exec_with_mask_0

April 15, 2025

An Exception was encountered at ‘In [1]’.

Execution using papermill encountered an exception here and stopped:

[1]:
```
import numpy as np
import torch
import tensorly as tl
from tensorly.decomposition import parafac
import sparse
import pickle
import os

assert os.path.exists('sptensor.pkl'), 'No such file.'
with open('sptensor.pkl', 'rb') as f:
    data = pickle.load(f)
data = data[:, :, :, :12, :]
expected_shape = data.shape
n_components = 10

# --------------------------------
# 3. NumPy-based BPTF (Aaron's original implementation)
# --------------------------------
# Import Aaron's BPTF (which uses NumPy/sparse.COO);
# ensure that the bptf package is in your PYTHONPATH.
from bptf import BPTF as BPTF
import bptf

# Create the same data as a NumPy array and a corresponding binary mask
data_np = data
mask_np = np.zeros(expected_shape, dtype=int)
mask_np[:, :, :, 3, :] = 1
mask_np = sparse.COO((1 - mask_np.copy()).astype(np.int64))

# Instantiate and fit the NumPy-based BPTF model.
# Note: This version uses its own preprocess() function and can work with␣
 ↪sparse.COO.
model_np = BPTF(data_shape=data_np.shape, n_components=n_components)
model_np.fit(data, mask=mask_np, max_iter=50, verbose=True, missing_val=0)
```

```
# Reconstruct using arithmetic expectation
reconstruction_np = model_np.reconstruct(mask=None, fill_value=1,␣
 ↪drop_diag=False, style='arithmetic')
frobenius_diff_np = np.sqrt(np.sum((data.todense() - reconstruction_np)**2))
print("NumPy BPTF reconstruction Frobenius norm difference:", frobenius_diff_np)
```

```
ITERATION 0:    Time: 0.000000  Objective: -29230329.71 Change: nan


  0%|
| 0/50 [00:00<?, ?it/s]

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 0]  min uttkrp_DK = 1.679e+05
[mode 0]  min new rate = 1.679e+05   negatives = 0
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 1]  min uttkrp_DK = 1.930e+03
[mode 1]  min new rate = 1.930e+03   negatives = 0
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 2]  min uttkrp_DK = 1.891e+04
[mode 2]  min new rate = 1.891e+04   negatives = 0
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 3]  min uttkrp_DK = 0.000e+00
[mode 3]  min new rate = 9.847e-02   negatives = 0
Is G_DK_M finite before updating? True
```

```
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 4]  min uttkrp_DK = 6.768e+04
[mode 4]  min new rate = 6.768e+04   negatives = 0
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True


  0%|
| 0/50 [01:04<?, ?it/s]

ITERATION 1:    Time: 64.709600 Objective: -714685487776.35     Change:
-2.44491e+04
```

```
---------------------------------------------------------------------------
AssertionError                            Traceback (most recent call last)
Cell In[1], line 33
     30 # Instantiate and fit the NumPy-based BPTF model.
     31 # Note: This version uses its own preprocess() function and can work␣
 ↪with sparse.COO.
     32 model_np = BPTF(data_shape=data_np.shape, n_components=n_components)
---> 33 model_np.fit(data, mask=mask_np, max_iter=50, verbose=True,␣
 ↪missing_val=0)
     35 # Reconstruct using arithmetic expectation
     36 reconstruction_np = model_np.reconstruct(mask=None, fill_value=1,␣
 ↪drop_diag=False, style='arithmetic')

File ~/projects/bptf_new/bptf/src/bptf/bptf.py:290, in BPTF.fit(self, data,␣
 ↪mask, missing_val, **kwargs)
    286     assert is_binary(mask)
    288 self._init()
--> 290 self._update(data, mask=mask, missing_val=missing_val, **kwargs)
    291 return self

File ~/projects/bptf_new/bptf/src/bptf/bptf.py:265, in BPTF._update(self, data,
 ↪mask, missing_val, modes, **kwargs)
    258     print ('ITERATION %d:\t'\
    259           'Time: %f\t'\
    260           'Objective: %.2f\t'\
    261           'Change: %.5e\t'\
    262           % (itn+1, e, bound, delta))
```

```
    264 # assert delta >= 0.0
--> 265 assert delta >= -1e-2
    266 curr_elbo = bound
    267 if delta < kwargs.get('tol', 1e-4):

AssertionError:
```

[ ]:
```
# ------------------------------
# 1. PyTorch-based BPTF (your version)
# ------------------------------
# Import your PyTorch-based BPTF model (adjust filename as needed)
from own_implementation import BPTF as BPTF_torch
tl.set_backend('pytorch')

device = 'cpu'
# Create a tensor from a Poisson distribution (counts) and a matching mask; ensure types match
data_torch = torch.tensor(data.todense(), dtype=torch.float64, device=device)
mask_torch = torch.ones(expected_shape, dtype=torch.float64, device=device)

# Instantiate and fit the PyTorch-based BPTF model
model_torch = BPTF_torch(data_shape=expected_shape, n_components=n_components, device=device)
model_torch.fit(data_torch, mask=mask_torch, max_iter=50, tol=1e-4, verbose=True)
reconstruction_torch = model_torch.reconstruct(mask=mask_torch, style='arithmetic')
frobenius_diff_torch = torch.norm(data_torch - reconstruction_torch, p='fro').item()
print("PyTorch BPTF reconstruction Frobenius norm difference:", frobenius_diff_torch)
```

[ ]:
```
# ------------------------------
# 2. TensorLy CP Decomposition
# ------------------------------
# Use TensorLy's parafac for CP decomposition (same rank as n_components)
cp_decomp = parafac(data_torch, rank=n_components, n_iter_max=100, init='random')
reconstruction_cp = tl.cp_to_tensor(cp_decomp)
frobenius_diff_cp = torch.norm(data_torch - reconstruction_cp, p='fro').item()
print("TensorLy CP decomposition Frobenius norm difference:", frobenius_diff_cp)
```